1 Build a perceptron model (on paper or spreadsheet) to solve a binary classification problem and apply the perceptron learning law.

Introduction to perceptron:

1.1 What is Perceptron?

The Perceptron is one of the simplest Artificial Neural Network (ANN) models introduced by

Frank Rosenblatt in 1958.

It is used for binary classification problems, meaning it classifies input into two classes:

1.0 or 1

2.Yes or No

3.True or False

1.2 Structure of Perceptron

A perceptron consists of:

Input values (x_1, x_2)

Weights (w_1, w_2)

Bias (b)

Activation function (Step function)

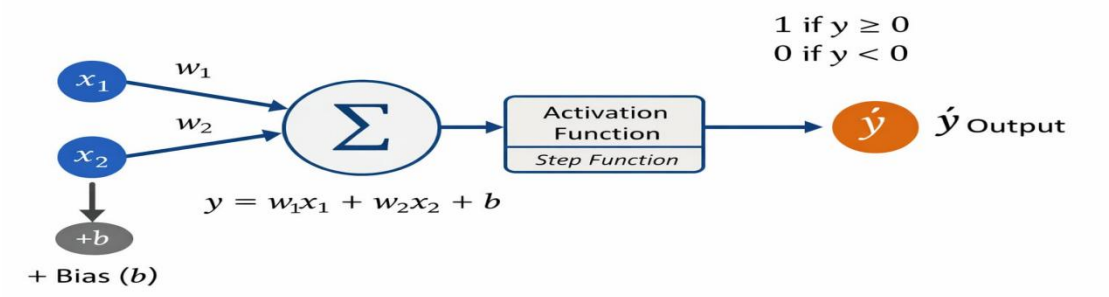
Mathematical Model:

y = w1x1 +w2x2+b

Output:

y={10 if z⩾0if z<0

1.3 Perceptron Architecture Daigram:

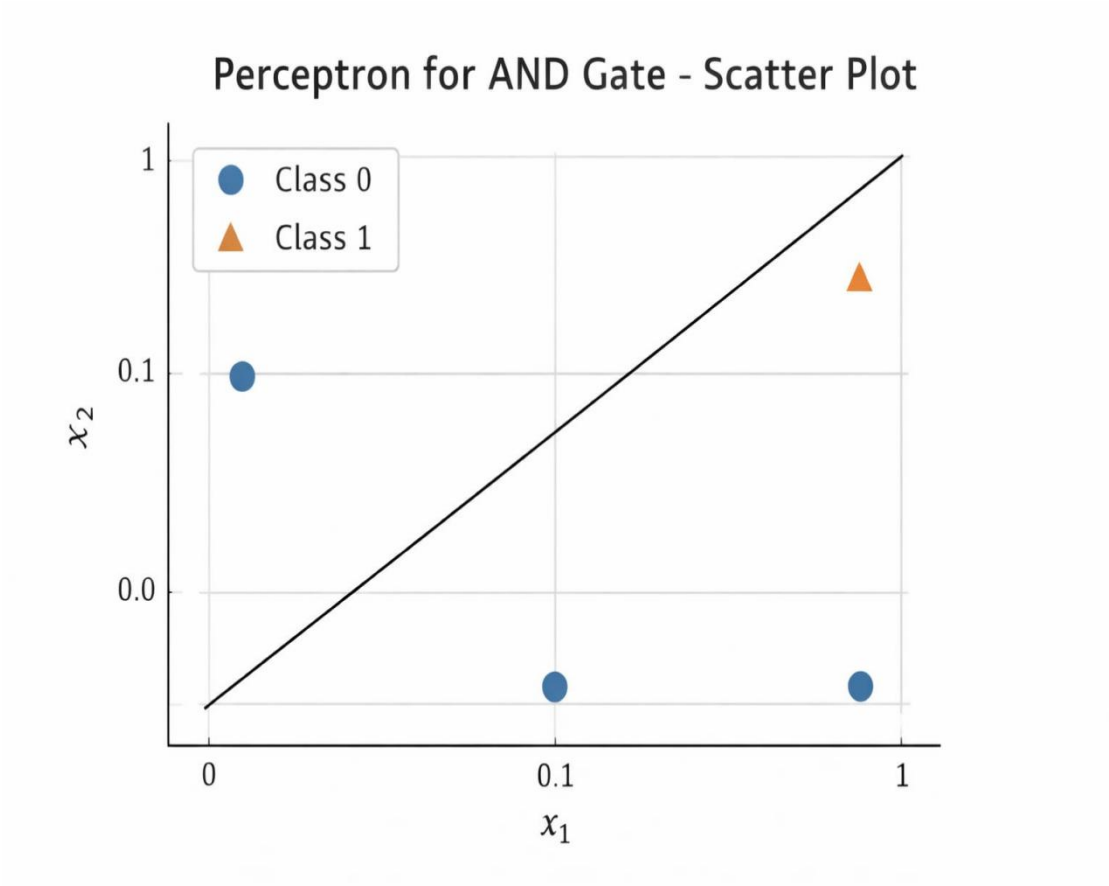


Problem Statement (AND Gate)

2.1 Binary Classification Example – AND Gate
We will implement Perceptron for AND gate.

x1	x2	Output (y)
0	0	0
0	1	0
1	0	0
1	1	1

2.2 Graphical Representation scatter plot:



Perceptron Learning Law

3.1 Learning Rule Formula

Net Input Equation

z=w· x+bz = \mathbf{w} \cdot \mathbf{x} + bz=w· x+b

z=w1x1+w2x2+…+wnxn+bz = w_1x_1 + w_2x_2 + \dots + w_nx_n + bz=w1 x1 +w2 x2 +…+wn xn +b

Learning Rule Formula

winew =wiold +η(t−y)xi

Where:

η = Learning rate (example 0.1)

T = Target output

ŷ = Predicted output

3.2 Step-by-Step Training (First Epoch Example)

Assume:

$w_1 = 0$

$w_2 = 0$

b = 0

η = 0.1

Input (1,1)

y = (0×1) + (0×1) + 0 = 0

ŷ = 1 (since y ≥ 0)

Target = 1

Error = T − ŷ = 1 − 1 = 0

No weight update needed.

You continue same process for all inputs until error becomes zero.

Spreadsheet Implementation:

Columns Required:

| x1 | x2 | w1 | w2 | b | net input | output | target | error | new w1 | new w2 |

Formula examples:

Net input = (w1x1 + w2x2 + b)

Error = Target - Output

New w1 = w1 + (0.1 * error * x1)

New w2 = w2 + (0.1 * error * x2)

Appendix: Python Implementation (Alternative Version)

```
import numpy as np

# AND Gate Dataset
inputs = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])


targets = np.array([0, 0, 0, 1])
w = np.random.randn(2)   # Random weight initialization
b = np.random.randn()    # Random bias
alpha = 0.5              # Learning rate
epochs = 15

def activation(net):
    return np.where(net >= 0, 1, 0)

for epoch in range(epochs):
    net_input = np.dot(inputs, w) + b
    predictions = activation(net_input)
    errors = targets - predictions

        w += alpha * np.dot(inputs.T, errors)
    b += alpha * errors.sum()

    print(f"Epoch {epoch+1}, Total Error = {np.sum(np.abs(errors))}")

print("\nTrained Weights:", w)
print("Trained Bias:", b)
print("\nFinal Predictions:")
final_output = activation(np.dot(inputs, w) + b


for i in range(len(inputs)):
    print(f"Input: {inputs[i]} → Output: {final_output[i]}")
```

Conclusion:

In this experiment, a Perceptron model was designed and implemented to solve a binary classification problem using the AND gate dataset. The objective was to apply the Perceptron learning law and understand how weights and bias are updated iteratively during training.

Initially, the weights and bias were assigned small values, and predictions were made using the step activation function.

During each epoch, the model calculated the net input as the weighted sum of inputs plus bias. The predicted output was compared with the target output, and the error was computed. Based on this error, the weights and bias were updated using the Perceptron learning rule:

$$w = w + \eta(T - \hat{y})x$$
$$b = b + \eta(T - \hat{y})$$

As training progressed, the total error gradually decreased and eventually became zero. This indicates that the perceptron successfully learned the correct decision boundary for the AND gate. Since the AND gate is a linearly separable problem, the perceptron was able to classify all input patterns correctly using a linear decision surface.

This experiment clearly demonstrates that the Perceptron algorithm is effective for solving linearly separable binary classification problems. It also provides practical understanding of supervised learning, weight adjustment, and convergence behavior in Artificial Neural Networks. However, it is important to note that the Perceptron cannot solve non-linearly separable problems such as XOR, which highlights the need for multi-layer neural networks.

Overall, the implementation validates both the theoretical concept of the Perceptron model and the correctness of the learning law through successful training and accurate predictions.